Module 4: End of Module Assignment

TaskFlow API - Dockerize & Deploy

Before you start: try, experiment and stretch yourself

This course is about gaining new skills, so we encourage you to try things out and experiment. If you get stuck or something fails at any point, **submit the work you have completed so far** and include the errors or failures you ran into in your `README.md`. Failure is part of the learning process.

The assignment is graded on a **pass/fail** basis, so every attempt is welcome. This means you can stretch yourself and try something ambitious without the fear of failing. **Stretch goals are encouraged** - see *Stretch Goals (Optional)* at the end of this assignment.

Overview

Type: Mandatory Assignment (graded pass/fail)

Estimated Time: 1-2 hours (plus extra time if you take on a stretch goal)

INTRODUCTION

You will take your TaskFlow API from Module 3 and containerize it using Docker. Then deploy it to Railway so it runs in production.

PREREQUISITES

You must have:

- Completed Module 3 assignment (TaskFlow API project)
- GitHub account
- Railway account (from Lesson 4.12)
- Docker installed on your computer

Your Tasks

TASK 1: CREATE DOCKERFILE

In your TaskFlow project root folder, create a file called: `Dockerfile`

Copy this code:

```
# Stage 1: Build the JAR with Maven
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY . .
RUN mvn clean package -DskipTests

# Stage 2: Run the JAR in a slim runtime image
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```


Copyright © 2026 Nanyang Technological University

Save the file.

Why a multi-stage build? The first stage builds the JAR inside Docker, so you don't need to build it on your computer first. This matters for Railway: it builds your image from the code in GitHub, and the `target/` folder is not pushed to GitHub (it is in `.gitignore`). The second stage copies only the JAR into a small runtime image. You learned this pattern in Lesson 4.4.

TASK 2: BUILD AND TEST DOCKER IMAGE LOCALLY

Open Terminal in VS Code.

Step 1: Build Docker image

```
docker build -t taskflow:latest .
```

The first build takes a few minutes while Maven downloads dependencies.

Step 2: Run the container

```
docker run -p 8080:8080 taskflow:latest
```

Step 3: Test the application

Open a second terminal and run:

```
curl http://localhost:8080/api/tasks
```

You should see JSON response with task data.

TASK 3: DEPLOY TO RAILWAY

Step 1: Push code to GitHub

In your project folder terminal:

```
git add .
git commit -m "Add Dockerfile for Module 4"
git push origin main
```

Step 2: Deploy on Railway

1. Go to https://railway.app
2. Click **"Start a New Project"**
3. Select **"Deploy from GitHub repo"**
4. Select your TaskFlow repository
5. Railway automatically detects your Dockerfile
6. Wait 2-3 minutes for deployment

Step 3: Generate public domain

1. Click on your service in Railway
2. Go to **Settings → Networking**
3. Click **"Generate Domain"**
4. Set port to `8080`

Copyright © 2026 Nanyang Technological University

5. Copy the generated URL

TASK 4: VERIFY DEPLOYMENT

Test your deployed application:

```
curl https://YOUR-RAILWAY-URL.up.railway.app/api/tasks
```

You should see JSON response from your live application!

Take a screenshot showing:

- Your Railway dashboard with active deployment
- Your live application URL working

You will add these screenshots to your `README.md` in the next step.

Submission

STEP 1: WRITE YOUR README.MD

Create (or update) `README.md` in the root of your repository. Everything we need to review your work goes in this file:

- **Live application URL** - your Railway URL (`https://...up.railway.app/api/tasks`)
- **Screenshots** - your Railway dashboard (deployment active) and your live application URL working. Save the images in your repository (for example, in a `screenshots/` folder) and show them in the README:

```
![Railway dashboard](screenshots/railway-dashboard.png)
```

- **Errors and failures** - if you didn't finish every task, list what you completed, what went wrong and the error messages you saw
- **Stretch goal** (optional) - which stretch goal you chose, what you built, and any links (for example, your frontend URL or CI pipeline)

STEP 2: PUSH TO GITHUB

Make sure your GitHub repo contains:

- `Dockerfile` in root folder
- `README.md` with your live URL and screenshots
- All TaskFlow source code

```
git add .
git commit -m "Add README with deployment details"
git push origin main
```

STEP 3: SUBMIT TO NTULEARN

Submit **only your GitHub repository URL** to NTULearn. Everything else is in your `README.md`.

Done! 🚀 Want more of a challenge? Try one of the stretch goals below.

Copyright © 2026 Nanyang Technological University

Stretch Goals (Optional)

Stretch goals are **optional**. They are here for anyone who wants to challenge themselves - you will not fail the assignment if you skip one, or if you try one and don't finish it. If you take one on, choose **one** of the options below and describe what you did (and any errors you hit) in your `README.md`.

OPTION 1: ADD A CI PIPELINE

Add a CI pipeline with **GitHub Actions** or **CircleCI** that builds, tests and publishes your TaskFlow API image to Docker Hub on every push to `main`. You will then have two pipelines working side by side: a **CI pipeline** to Docker Hub and a **deployment pipeline** to Railway.

- Refer to Lesson 4.7 (CircleCI) or Lesson 4.9 (GitHub Actions)
- Store your Docker Hub credentials as secrets - never in your code
- In your README, include a link to a successful pipeline run and your image on Docker Hub

OPTION 2: ADD A FRONTEND ON NETLIFY

Create a frontend that uses your TaskFlow API and deploy it to **Netlify**, with your backend still running on Railway.

- The frontend should call your **Railway** API URL (for example, list your tasks)
- Your browser will block requests from the Netlify domain until you allow it in your API: look up **CORS** in Spring Boot
- In your README, include your Netlify URL and a screenshot of the frontend showing data from the API

OPTION 3: USE YOUR SIMPLE CRM PROJECTS

Instead of TaskFlow, containerize and deploy the **Simple CRM API** backend you built in Module 3 to Railway, and deploy the **Simple CRM React frontend** from Module 2 to Netlify, connected to your Railway API.

- Complete Tasks 1-4 using your Simple CRM API instead of TaskFlow (test your own endpoints instead of `/api/tasks`)
- If your API uses a database, add a database on Railway and connect it with environment variables (see Lesson 4.10)
- In your README, include both live URLs and screenshots of the frontend showing data from the API

OPTION 4: PUBLISH VERSIONED DOCKER IMAGES

Tag your TaskFlow API images with version numbers instead of only `latest`, and push them to Docker Hub.

- Use Semantic Versioning (`MAJOR.MINOR.PATCH`, e.g. `1.0.0`) from Lesson 4.5, and/or the Git commit SHA
- Push at least two versions, and keep `latest` pointing at the newest one
- Bonus: tag the images automatically in a CI pipeline (Option 1)
- In your README, explain your versioning scheme and include a screenshot of the tags on Docker Hub

OPTION 5: ADD SECURITY CHECKS

Add security checks to your TaskFlow API using what you learned in Lessons 4.13 and 4.14.

- Scan your dependencies for known vulnerabilities with OWASP Dependency-Check (Lesson 4.14)
- Optionally, also scan your Docker image for vulnerabilities with a container scanner
- Run the checks locally, or better, in a CI pipeline (Option 1)
- Make sure no passwords, tokens or other secrets are committed to your repository
- In your README, summarise what the scans found and what you fixed (or why you left an issue as it is)

Copyright © 2026 Nanyang Technological University